Cybersecurity Interview Handbook

Q1. What is authentication?

Authentication is the process of verifying who a user or system is.

Example:

When you log in to Gmail using:

Email + Password

the system verifies that you are the person associated with that account.

Q2. What is authorization?

Authorization determines what an authenticated user is allowed to do.

Example:

Suppose you're logged into a college portal.

Student → View marks
Teacher → Enter marks
Admin → Add/remove users

Everyone may be authenticated, but their **permissions are different**.

Q3. What is the difference between authentication and authorization?

A comparison of verifying identity versus managing permissions.

Authentication	**Authorization**
Verifies identity	Determines permissions
"Who are you?"	"What can you do?"

Example:

Imagine entering an office:

Security guard checks your ID
↓
Authentication
↓
You're allowed into the building

↓
Access control checks which rooms you can enter
↓
Authorization

Easy way to remember:

Authentication = Who are you?
Authorization = What can you do?

Q4. What is encryption?

Encryption converts readable data (plaintext) into unreadable data (ciphertext) using a cryptographic algorithm and key.

Example:

When sensitive information is sent over an encrypted connection, an attacker who intercepts the traffic should not be able to read the plaintext.

Q5. What is hashing?

Hashing converts input data into a fixed-length hash value using a one-way hash function.

Example:

"hello"
↓
Hash function
↓
some fixed-length hash

If you change the input:

"Hello"

the resulting cryptographic hash will be dramatically different.

Important:
For passwords, applications should use a **password hashing/KDF algorithm** such as Argon2id, bcrypt, scrypt, or PBKDF2, with a unique salt—not a plain fast hash like SHA-256 by itself.

Q6. What is the difference between encryption and hashing?

Encryption is reversible and protects confidentiality, while hashing is one-way and protects integrity.

Encryption	Hashing
Confidentiality / Reversible	Integrity / One-way

Example:

For a message:

"Hello Rahul"

Encryption:

Message → Encrypted data → Decrypt → Message

Hashing:

Message → Hash

You don't normally "decrypt" a hash to get the original message.

Q7. What is symmetric encryption?

Symmetric encryption uses the same secret key for both encryption and decryption.

Examples:

Common symmetric algorithms include:

AES
ChaCha20

Real-world example:
Imagine a box that can be locked and unlocked using the **same key**. Both parties need access to that secret key.

Q8. What is asymmetric encryption?

Asymmetric cryptography uses a pair of related keys: a public key for encryption and a private key for decryption.

Examples:

RSA
Elliptic-curve cryptography (ECC)

Real-world example:

Imagine a mailbox:

Public key → anyone can use it to put something in
Private key → only the owner can use it to access the protected contents

That's a simplified analogy for one common use of public-key encryption.

Q9. Symmetric vs Asymmetric encryption?

Symmetric is faster but has key-sharing challenges; asymmetric is slower but easier to distribute.

Symmetric	**Asymmetric**
One shared secret key	Public/private key pair

Important real-world point:
HTTPS generally uses **both**.

A simplified view is:

Asymmetric/public-key cryptography
↓
Establish/authenticate secure session
↓
Symmetric session keys
↓
Encrypt bulk application data

This gives us the benefits of both approaches.

Q10. What is HTTPS?

HTTPS is HTTP carried over a TLS-protected connection, providing confidentiality, integrity, and authentication.

HTTPS normally uses:

Port 443

Example:

When you visit:

https://example.com

your HTTP communication is protected by TLS.

Q11. What is SSL/TLS?

TLS (Transport Layer Security) is the modern protocol used to secure network communications; SSL is its obsolete predecessor.

SSL was the older predecessor to TLS and is obsolete.

So in modern systems, you should say:

> **"HTTPS uses TLS; SSL is an older, deprecated protocol."**

Example:

When you log into a website using HTTPS:

Browser
↓
TLS-protected connection
↓
Server

Your password and other HTTP data are protected while traveling across the network.

Q12. What is a digital signature?

A cryptographic mechanism used to verify the authenticity and integrity of data.

The receiver can use the corresponding public key to verify the signature.

Example:

Suppose a company digitally signs a software update.

The user can verify:

Was this signed by the expected publisher?
Has the file been modified?

If the file changes, signature verification should fail.

Important distinction:

> A digital signature is **not the same as encryption**.

A signature is primarily about **authenticity/integrity**, whereas encryption is primarily about **confidentiality**.

Q13. What is SQL Injection?

A vulnerability where untrusted input changes the logic of an SQL query, prevented primarily by using parameterized queries.

Example:

Imagine a login form:

Username: user input
Password: user input

If the application directly inserts those values into SQL without proper parameterization, an attacker may manipulate the query logic.

How do we prevent it?

The main defense is:

> **Use parameterized queries / prepared statements.**

For example:

```
PreparedStatement stmt =
  connection.prepareStatement(
    "SELECT * FROM users WHERE username = ?"
  );

stmt.setString(1, username);
```

The database treats the supplied value as **data**, rather than allowing it to become SQL syntax.

Other defenses include appropriate input validation and least-privilege database accounts.

Q14. What is XSS?

Cross-Site Scripting occurs when malicious scripts execute in a user's browser via a trusted website.

Suppose a website allows users to post comments.

A user enters malicious content instead of normal text.

If the application renders it unsafely, another user's browser may execute the injected script.

How do we prevent XSS?

Common defenses include:

- Context-appropriate output encoding
- Safe HTML templating
- Sanitizing HTML when HTML is intentionally allowed
- Content Security Policy (CSP)
- Avoiding unsafe DOM APIs such as careless use of innerHTML

Real-world example:

A comment section should display:

Hello everyone!

as text—not interpret user-provided content as executable browser code.

Q15. What is CSRF?

Cross-Site Request Forgery tricks a browser into sending unwanted authenticated requests.

Suppose you're logged into your bank.

Then you visit a malicious website.

That site attempts to cause your browser to send an unwanted request to the bank, such as transferring money.

If the bank doesn't have appropriate CSRF defenses, the request might be accepted as coming from your authenticated browser.

How do we prevent CSRF?

Common defenses include:

- CSRF tokens
- Appropriate SameSite cookie settings
- Checking request origin/referer where appropriate
- Designing APIs with appropriate authentication mechanisms

Easy way to remember:

> **CSRF tricks your browser into making a request you didn't intend to make.**

Q16. What is a firewall?

A security system that controls network traffic based on predetermined security rules.

Example:

Suppose a server should accept HTTPS traffic but shouldn't expose SSH to the public Internet.

A firewall can have rules conceptually like:

Internet
↓
Firewall
├── Port 443 → Allow
└── Port 22 → Block/restrict

So unwanted network connections can be blocked.